Object-Oriented Programming

1. Introduction

Filip Křikava

https://courses.fit.cvut.cz/BI-OOP/

Out of all the monsters, which one is the worst?



https://www.comicartfans.com/gallerypiece.asp?piece=1041716

"Of all the monsters ..., none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, one seeks bullets of silver that can magically lay them to rest.

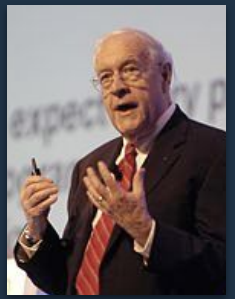
The familiar software project, at least as seen by the nontechnical manager, has something of this character; it is usually innocent and straightforward, but is capable of becoming a monster of missed schedules, blown budgets, and flawed products. So we hear desperate cries for a silver bullet--something to make software costs drop as rapidly as computer hardware costs do.

But, as we look to the horizon of a decade hence, we see no silver bullet. There is no single development, in either technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity.

In this article, I shall try to show why, by examining both the nature of the software problem and the properties of the bullets proposed." -- **Fred Brooks, No Silver Bullet**, Computer 1986

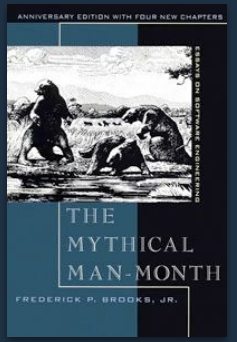
Intermezzo - Fred Brooks's No Silver Bullet

Essence and Accidents of Software Engineering, 1986



Fred Brooks

- Essential Complexity
  - Intrinsic to the problem to be solved
  - Nothing can remove it
  - *e.g.* if a program must do 30 things, they are all essential and must be implemented
- Accidental Complexity
  - Caused by the approach one has chosen
  - Can be reduced
  - *e.g.* using better suited programming language or methodology
- Brook's law
  - Adding manpower to a late software project makes it later.



https://dl.acm.org/citation.cfm?id=26441

Intermezzo - Fred Brooks's No Silver Bullet

Essence and Accidents of Software Engineering, 1986

There is no single development, in either technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity

-- **Fred Brooks, No Silver Bullet**, Computer 1987

Intermezzo

Computer Science

Intermezzo

Computer Science has always been about one thing:

Translating ideas into programs.

CS is the study of how to take a problem and map it onto instructions that can be executed by a Von Neumann machine.

Intermezzo

Critically, the goal of CS has always been that programs are implemented, maintained, and understood by humans.

Intermezzo

Fifty years of programming language research has done nothing to improve the state of affairs.

No amount of improvement to type systems, debugging, static analysis, linters, or documentation is going to magically solve this problem.

*** COMPUTER SCIENCE IS DOOMED ***

Intermezzo



// Large language models
// and the end of programming

// Matt Welsh <matt@fixie.ai>
//
// February 2023

https://www.youtube.com/watch?v=s0ZeaUY9a3k

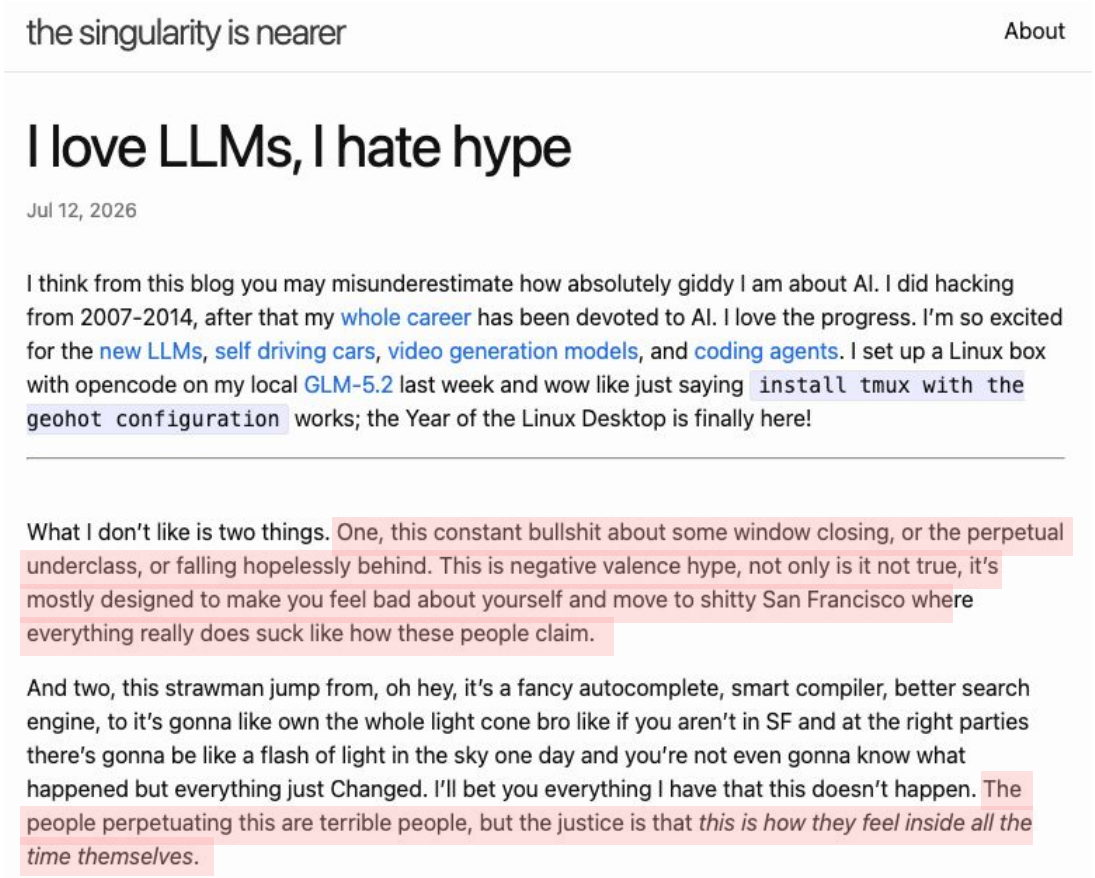
Communications of the ACM, January 2023
https://dl.acm.org/doi/pdf/10.1145/3570220

The New York Times magazine, March 12. 2026 by Clive Thompson

Coding After Coders: The End of Computer Programming as We Know It

In the era of A.I. agents, many Silicon Valley programmers are now barely programming. Instead, what they're doing is deeply, deeply weird.

Clive Thompson interviewed more than 70 software developers at Google, Amazon, Microsoft and small start-ups. He is the author of "Coders: The Making of a New Tribe and the Remaking of the World."



the singularity is nearer About

I love LLMs, I hate hype

Jul 12, 2026

I think from this blog you may misunderestimate how absolutely giddy I am about AI. I did hacking from 2007-2014, after that my whole career has been devoted to AI. I love the progress. I'm so excited for the new LLMs, self driving cars, video generation models, and coding agents. I set up a Linux box with opencode on my local GLM-5.2 last week and wow like just saying `install tmux with the geohot configuration` works; the Year of the Linux Desktop is finally here!

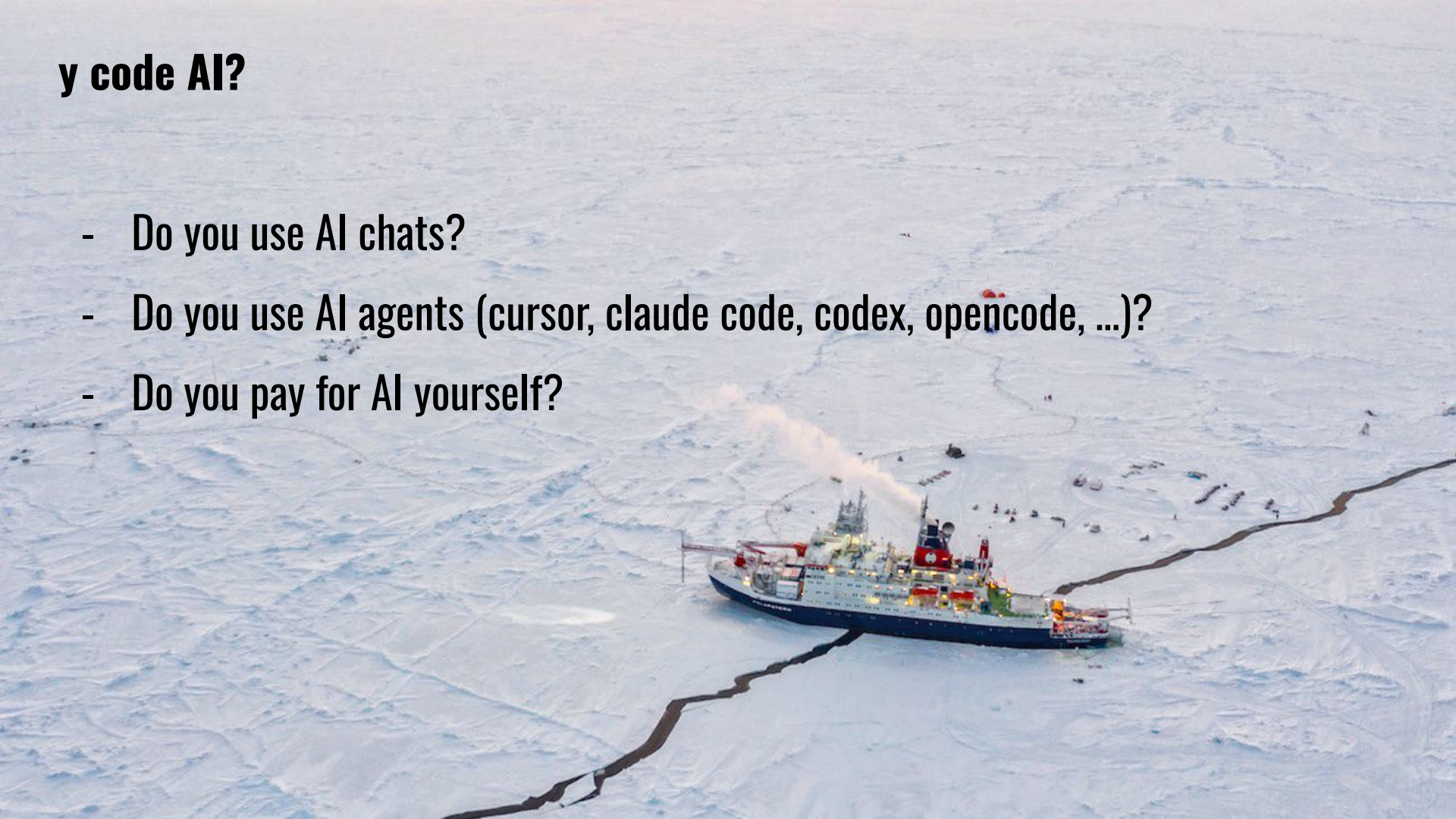
What I don't like is two things. One, this constant bullshit about some window closing, or the perpetual underclass, or falling hopelessly behind. This is negative valence hype, not only is it not true, it's mostly designed to make you feel bad about yourself and move to shitty San Francisco where everything really does suck like how these people claim.

And two, this strawman jump from, oh hey, it's a fancy autocomplete, smart compiler, better search engine, to it's gonna like own the whole light cone bro like if you aren't in SF and at the right parties there's gonna be like a flash of light in the sky one day and you're not even gonna know what happened but everything just Changed. I'll bet you everything I have that this doesn't happen. The people perpetuating this are terrible people, but the justice is that *this is how they feel inside all the time themselves*.

https://geohot.github.io/blog/jekyll/update/2026/07/12/i-love-llms.html

y code AI?

- Do you use AI chats?
- Do you use AI agents (cursor, claude code, codex, opencode, ...)?
- Do you pay for AI yourself?



Agenda

1. Administrative trivia
2. Fundamentals of OOP

The use of AI in BI-OOP

- Use it, but use with care
  - Think about what you want to learn (the AI has already learn it)
  - Think of it as super-powered classmate
    - it is OK to ask for help, code review, suggestions, it is not OK to ask for writing the entire project
  - It is great for
    - Helping with the new programming language
    - Explaining error messages
    - Bootstrapping test suite
      - … but make sure that it makes sense, useless / duplicated tests do not count
    - Documentation
      - … but make sure it is correct
- CTU offers some AI
  - https://help.fit.cvut.cz/en/services/ai/index.html

Administrative trivia

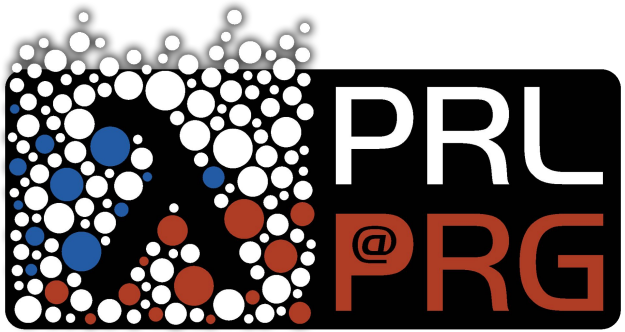
- All communications on Team-BI-OOP
- All info on https://courses.fit.cvut.cz/BI-OOP/
  - Schedule
  - Classification

@czech-speaking-students: as long as there is space, feel free to attend the English lectures / labs.

@all: if you spot a mistake, typo or just something does not make sense, please submit an issue on courses.

PRL@PRG lab

https://prl-prg.github.io/student-projects.html



- Founded in fall 2016 by prof. Jan Vitek
- Research in programming languages

- Bachelor & master thesis possibilities!!
- Internship possibilities!!

Past bachelor's projects:

- Haskell dynamic tracing
- Dynamic test generation for R packages
- Pattern matching in C11
- An actor model implementation for the OCaml programming language
- Analyze Scala code using graph database
- Simple Object Machine implementation in functional programming language
- An overview of gradual typing approaches in dynamic programming languages
- Copy-and-patch just-in-time compiler for the R programming language



https://prgprg.org

PRG · PRG

Prague Programming Languages and Systems Research Network

We are a group of researchers based in Prague interested in programming languages and systems from the Programming Research Laboratory at the Czech Technical University and the Department of Distributed and Dependable Systems at the Faculty of Mathematics and Physics, Charles University. We meet regularly, organise joint events and promote programming languages and systems research, projects and courses in Prague.

If you are interested in joining our network, drop us an email or come to any of the events listed below!

Regular Events

We run a number of events at both Czech Technical University and Charles University where we can meet and talk about programming languages and systems. To make talking to each other easier, our reading group alternates between the two locations!

Reading Group

Wednesdays (TBD), Malá Strana or Dejvice
List of past and upcoming meetings

Group Lunch at MFF

Mondays 13:00, Café HAMU, Malá Strana
Come and loook for us or email Tomas to check

Administrative trivia - schedule

	Date	Lecture
1.	25. 9.	Principles of object-oriented programming
2.	2.10.	Subtyping polymorphism
3.	9.10.	Agenting systems and agentic software development (Ondřej Rozinek)
4.	16.10.	Parametric polymorphism
5.	23.10.	Adhoc polymorphism
6.	30.10.	Composition and inheritance
7.	6.11.	Error Handling in OOP
8.	13.11.	OOP design patterns (Jan Kurš)
9.	20.11.	Testing in OOP
10.	27.11.	Reflection and object model
11.	4.12.	Pure OOP langauges (Sebastian Krynski)
12.	11.12.	Prototype-based inheritance (Petr Máj)

Administrative trivia - classification

- **Assessment**
  - **Mid-term test**
    - min 10, max 20 points
    - 10 questions, each worth 2 points
    - in person, on paper
    - covers lectures 1-6
    - retake 13. week
    - sample test
  - **Project**
    - min 25 (must be without the bonus), max 50 points (even with the bonus)
    - short in person , veto power
    - **all details on project page**
- **Exam**
  - min 15, max 30 points
  - 10 questions including reading and writing short snippets of Scala code
  - in person, on paper
  - covers all lectures
  - *optional* oral exam to improve your grade, to up to 10 points, veto power.
  - sample exam

Date	Item
10.11.2026	Mid-term
27.11.2026 6pm	Project soft-deadline +6 points
4.12.2026 6pm	Project soft-deadline +3 points
15.12.2026	Mid-term retake
18.12.2026 6pm	**Project hard-deadline**

Administrative trivia - project

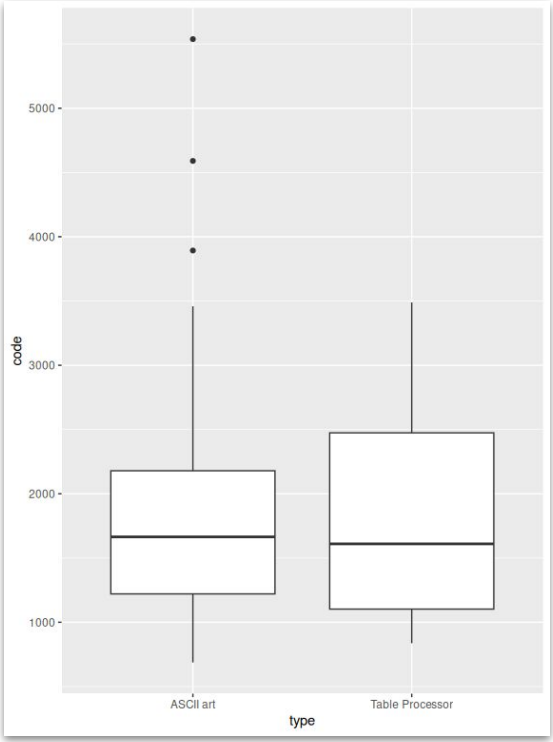
- Main part of the grade is a project
- Two projects to choose from
- The idea is that you can experiment with OOP on somewhat a larger scale
- More information at the labs

Administrative trivia - project

- **The project takes time**
  you won't be able to finish over the weekend!

- **The deadline is firm**
  no mercy with late excuses!

  If you see that you have a hard time, schedule a consultation with a TA or me (we are here to help). The only problem is the problem we do not know about, everything else is workable.




The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!
The deadline is firm! No late excuses!

Disclaimer

This class is hard to teach.

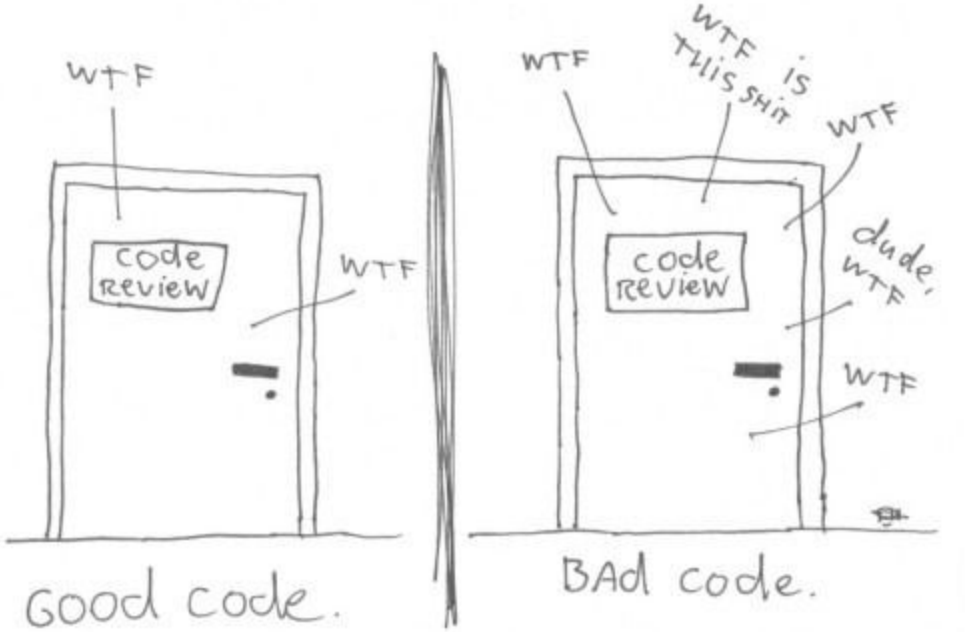


The reason is ...

somewhat harder to explain, please stay with me

Have you ever been slowed down by bad code?

The ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE



https://www.osnews.com/story/19266/wtfsm/



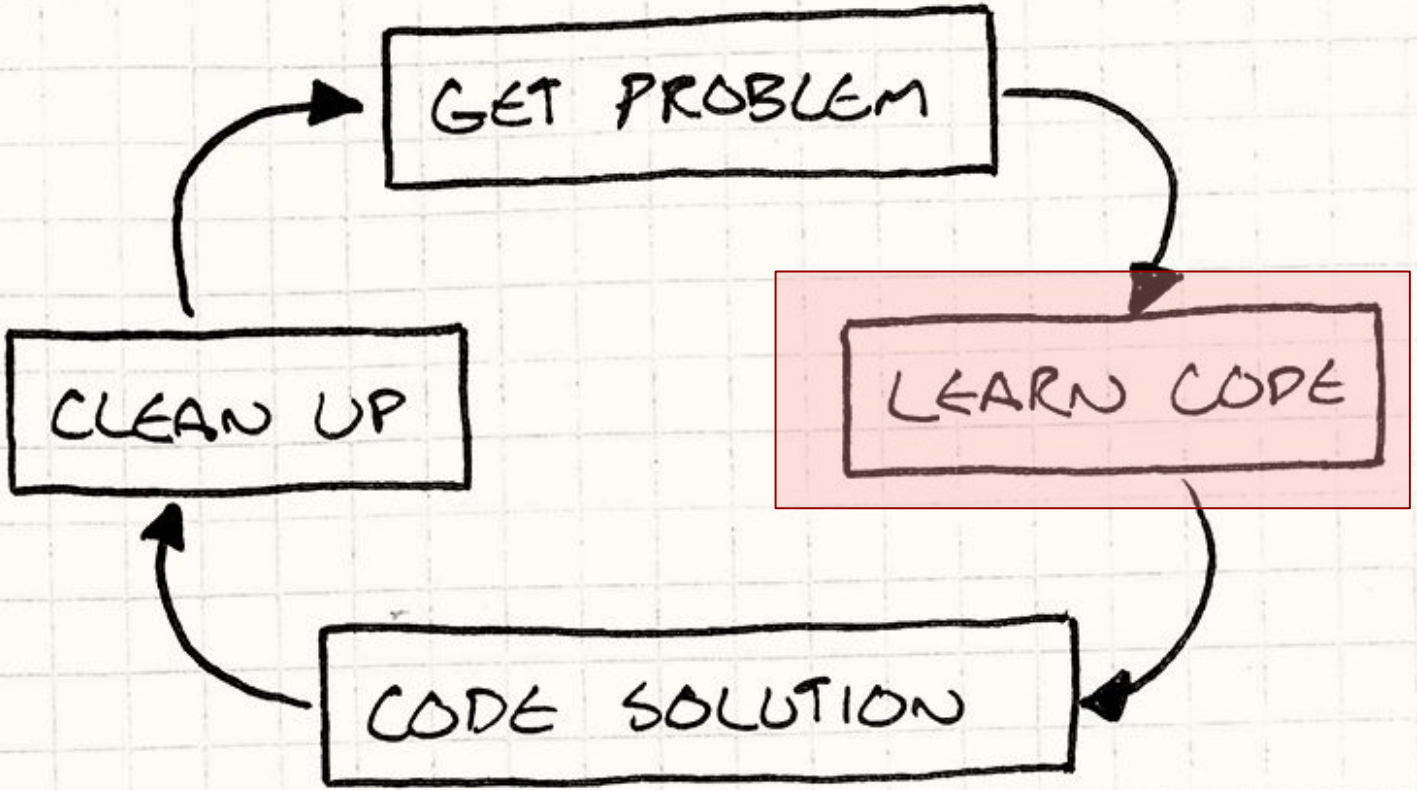
https://www.menshealth.com/uk/building-muscle/a755460/fight-club-an-oral-history/


The first rule about software design is that it is all about change.
If no one needs to touch the code (because it is perfect/finished/obsolete), its design is irrelevant.
https://www.menshealth.com/uk/building-muscle/a755460/fight-club-an-oral-history/

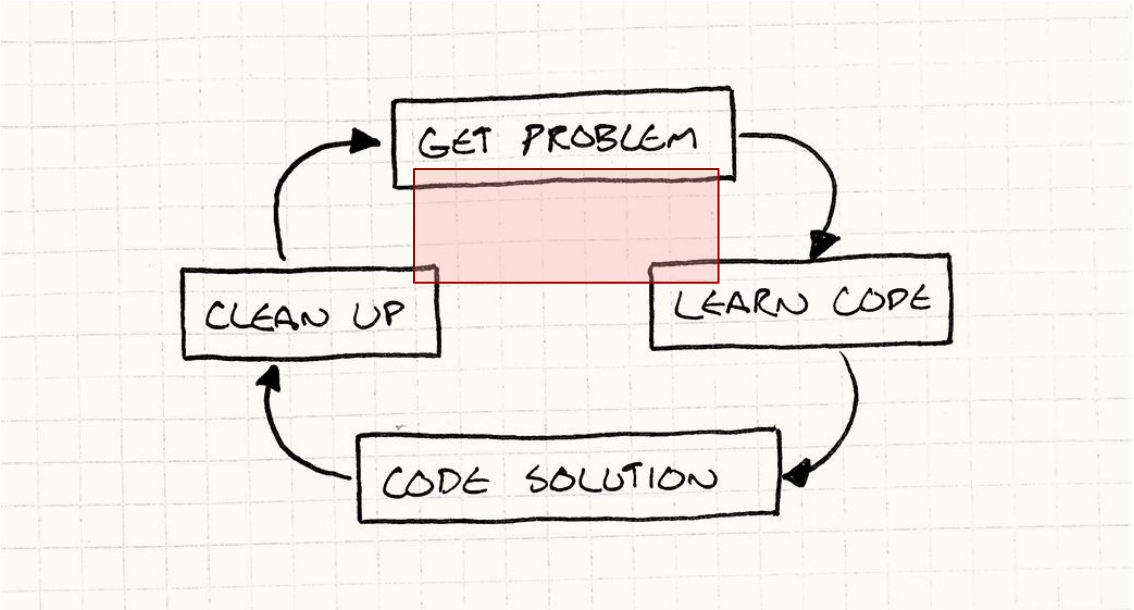
Good design is about

how well it accommodates change.

Now, how do you make a change?



https://gameprogrammingpatterns.com/architecture-performance-and-games.html



- Understanding code is slow.
- We need strategies to reduce the volume of code we need to understand at once.
- Who is the biggest enemy of the code volume reduction?

https://gameprogrammingpatterns.com/architecture-performance-and-games.html

Coupling

- Two pieces of code are coupled, it means you can't understand one without understanding the other.

- A change to one piece of code doesn't necessitate a change to another.

We obviously need to change something, but the less coupling we have, the less that change ripples throughout the rest of the code.

-- Robert Nystrom, Game programming patterns, 2011

Coupling

- Much of software design is managing dependencies
- Organizing what goes where so the dependencies do not run to strange and unexpected places.
- Software architecture is not that much about functionality, but rather about code organization in such a way that allows maintenance.

Good design is about

how well it accommodates change.

Minimize the amount of knowledge you need to have before you can make progress.


FREE
LUNCH
$10.00
SCHWADRON

The cost of decoupling



- price of abstraction
  - it is essentially a speculation that you will need the flexibility later
  - it delays from the core problem
- price of performance
  - every new layer of abstraction can (and usually does) introduce a performance penalty
- price of maintenance
  - not only to organize the code
  - but also to keep it organized

Recommendation

1. Make it work
    - always have something that is demoable (demo-oriented programming)
2. Make it right
    - once you are happy that it works, you have enough tests, refactor to make it right
3. Make it fast
    - optionally, if you have proof that it needs performance tuning, go for it

https://wiki.c2.com/?MakeItWorkMakeItRightMakeItFast

What goes wrong with software?





The key things about software design is that it is all about change.

If all you do is a few hundreds line of code for a one-off homework, you couldn't care less! You will only start appreciating when it grows and you have to go through the pain of refactoring.

https://www.menshealth.com/uk/building-muscle/a755460/fight-club-an-oral-history/

The reason is ...

somewhat harder to explain, please stay with me

... teaching pain is difficult.

Symptoms of Rotting Design

- Rigidity
  - the tendency for software to be difficult to change, even in simple ways
  - every change causes a cascade of subsequent changes in dependent modules
- Fragility
  - the tendency of the software to break in many places every time it is changed
  - often the breakage occurs in areas that have no conceptual relationship with the area that was changed
- Immobility
  - inability to reuse software from other projects or from parts of the same project
- High design viscosity
  - when the design preserving methods is harder to employ than the hacks
  - it is easy to do the wrong thing, but hard to do the right thing
- High environment viscosity
  - when the development environment is slow and inefficient
  - *e.g.* slow compile time, making engineers to do smaller changes which are bad for design

Goals of BI-OOP

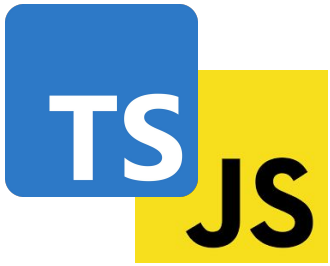
- What is this course about
  - main concepts of OOP and how to use them efficiently to design and implement software
  - study polymorphism (subtyping, parametric and adhoc)
  - how to use immutable data structure and combine FP with OOP
  - look at composition, inheritance and encapsulation
  - look at error handling and testing

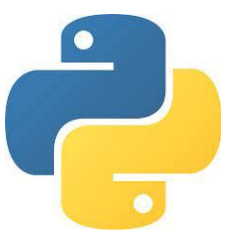
- What is this course not primarily about
  - Scala (*cf.* NI-PSL)
  - Design patterns (*cf.* NI-ADP)

We need some vehicle to embark on the OOP journey, so as a side-effect you will learn some Scala, but in lectures we will draw examples from other languages.

Foundations of Object-Oriented Programming

Most of you had been exposed to some OOP, right?













y and FP?

- Do you like functional programming?



What is Object-Oriented Programming?

What is an Object?

Data abstraction

- What is data abstraction? *Any mechanism that hides how data is implemented. The question is what, exactly, gets hidden.*
- What is abstract data type?
- What is object?

- Are they the same?
- Is one special kind of the other?

TL;DR

Objects and abstract data types are two different forms of data abstraction.

Neither is a variant of the other.

They are complementary: the strengths of one are the weaknesses of the other.

Running example: sets of integers

```
type set

empty     :  set
isEmpty   :  set → bool
insert    :  set × int → set
contains  :  set × int → bool
union     :  set × set → set
```

- Five operations: one constructor, two observations, two that build new sets.
- Keep your eye on union. It is the only operation that touches two sets at once.
- Such an operation is called complex — or, in most of the literature, binary.
- Almost everything interesting in this lecture follows from union.

Abstract data type in C

set.h

```
#ifndef SET_H
#define SET_H
#include <stdbool.h>

/* A name. The representation is NOT here. */
typedef struct set_rep *set;

set  empty(void);
bool is_empty(set s);
set  insert(set s, int i);
bool contains(set s, int i);
set  set_union(set s1, set s2);

#endif
```

- What the client sees, the set interface: one type name and five operations
- `struct set_rep` is declared and never defined in the header, an opaque pointer.
- The client only needs the size of a pointer, not the layout behind it.
- The header is the interface. The .c file is the implementation, i.e. C's module system of two files and a convention.
- This is the classic C idiom for an abstract data type heavily used.

The hidden representation

set.c

```c
#include <stdlib.h>
#include "set.h"

struct set_rep { int head; struct set_rep *tail; };

static set cons(int h, set t) {
  set n = malloc(sizeof(struct set_rep));
  n->head = h; n->tail = t; return n;
}

set  empty(void)     { return NULL; }
bool is_empty(set s) { return s == NULL; }

bool contains(set s, int i) {
  for (; s != NULL && s->head <= i; s = s->tail)
    if (s->head == i) return true;
  return false;
}

set insert(set s, int i) {
  if (s == NULL || i < s->head) return cons(i, s);
  if (i == s->head)             return s;
  return cons(s->head, insert(s->tail, i));
}
```

- One representation type. Every set in the whole program is a sorted list.
- Each operation takes a set as an argument and may look inside it freely.
- Sorting is an implementation decision: it makes contains and union faster. The client never learns about it.
- Change the representation to a hash table or a balanced tree and no client code changes, i.e. **representation independence**.

`union` inspects two representations at once

set.c

```c
/* Linear-time merge of two sorted lists. */
set set_union(set s1, set s2) {
  if (s1 == NULL) return s2;
  if (s2 == NULL) return s1;

  if (s1->head < s2->head)
    return cons(s1->head, set_union(s1->tail, s2));
  if (s1->head > s2->head)
    return cons(s2->head, set_union(s1, s2->tail));

  return cons(s1->head, set_union(s1->tail, s2->tail));
}
```

- `s1->head` and `s2->head` in the same expression.
- That is what makes the linear-time merge possible.
- The type system guarantees that every set value came from this implementation.
- The ability to inspect several representations at once is an advantage of abstract data types.

What makes this an abstract data type

- A public name, a hidden representation, and operations to create, combine and observe values.
- The hiding mechanism is type abstraction, enforced by the compiler.
- There is no equality unless you write one and publish it.

```
SetImp  =  ∃ rep . {
     empty    : rep,
     isEmpty  : rep → bool,
     insert   : rep × Int → rep,
     contains : rep × Int → bool,
     union    : rep × rep → rep
}
```

Formally, set is an existential type: *there exists a type, locally called rep, such that these operations are defined*.

ADT cost 1: one implementation per program

```
/* set.h    — sorted linked list  */
typedef struct set_rep  *set;

/* set2.h   — balanced tree       */
typedef struct set2_rep *set2;

set  set_union (set  a, set  b);
set2 set2_union(set2 a, set2 b);

/* …and now: */
set u = set_union(a, b2);   /* ??? */
```

- The type name set is globally bound to one hidden representation.
- Two implementations of the same abstraction clash on the name; one of them has to be renamed.
- Even where two can coexist, their values cannot be mixed: each opening of an existential creates a fresh type.

Not merely illegal — inexpressible

There is no union that takes a set and a set2. Given these signatures, you cannot even write its type.

The price, part 2: no new kinds of set

How would you add “the set of all even integers” to the C version.

1. Tag the representation

`struct set_rep` now needs a variant: a list, or a predicate, or an interval…

The layout of every existing set changes.

2. Edit every operation

`contains`, `insert`, `is_empty` and `union` each gain a branch per variant.

`union` gains one branch per pair of variants.

3. Recompile the world

Everything that links against `set.c` is rebuilt, including code you do not own.

New operations are cheap, new kinds of value are not.

Abstract data types have a protected representation type.

That is precisely what forbids extension.

The other way around

What if a set simply is a characteristic function?

```
type ISet = Int => Boolean

val Empty = (i: Int) => false

def Insert(s: ISet, n: Int): ISet =
  i => i == n || s(i)

def Union(s1: ISet, s2: ISet): ISet =
  i => s1(i) || s2(i)

val a = Insert(Empty, 1)
val b = Insert(a, 3)
b(3)    // true
```

- The empty set is the function that always says no.
- Inserting n builds a function that tests for n, or asks the old set.
- Union builds a function that asks both.
- There is no contains operation, the set is the contains operation.
- Nothing is hidden behind a type here.
- Everything is hidden behind a function.

Pure objects

Objects are essentially higher-order functions passed around!

From one function to a record of functions

The object-oriented interface to integer sets.

```
record ISet(
  isEmpty:  Boolean,
  contains: Int => Boolean,
  insert:   Int => ISet,      // ISet again
  union:    ISet => ISet      // ISet again
)
```

```
ISet  =  μ t . { isEmpty  : Bool,
                 contains : Int → Bool,
                 insert   : Int → t,
                 union    : t → t
}
```

- A record type whose fields are functions. In object-oriented vocabulary: an interface, and its fields are methods.
- No type abstraction anywhere: Boolean, Int and ISet are all public. The hiding is procedural.
- The type is recursive — insert and union mention ISet. That recursion is the μ binder.
- The language often lets you write the μ implicitly, by giving the record a name. In the λ-calculus you must write it out.

Language - let's design a simple language

```
record Counter(            -- a record type                    values: int, bool, functions and records
  get: () => Int,
  inc: () => Unit
)

Counter(get = g, inc = i)  -- a recordvalue
c.get                      -- field projection

x => e        a function           f(a)          application
() => e       a thunk              t()           forces a thunk
name = e      binds a name         { e1; e2 }    a block, value is e2
if (e1) e2 [else e3] condition     e1 op e2      basic arithmetics (+,-,*,/) and logic (&&,||)

record CounterRep { var x: Int }   -- a record holding a mutable cell
r.x = r.x + 1                      -- assignment
```

The rule: structural subtyping

{ get, inc, reset } <: { get, inc }

A record with more fields may be used wherever one with fewer is expected — silently, and at no run-time cost.

Implementation of the empty set

```
ISet  =  μ t . { isEmpty  : Bool,
                 contains : Int → Bool,
                 insert   : Int → t,
                 union    : t → t }
```

```
Empty = μ this. { isEmpty  = true,
                  contains = λi. false,
                  insert   = λi. Insert(this, i),
                  union    = λs. s }
```

- How could we implement the μ binder? How to implement the recursion?
- μ x . f is the value of f in which every occurrence of x denotes the value itself.
- **this** is Smalltalk's and Python's self, C++'s this, Java's this, ...

Recursion without a name (recap)

Take the λ-calculus, add numbers and a conditional, and try to write factorial.

What we want:

$$fact = \lambda n.\ n \leq 1\ ?\ 1 : n * fact\ (n\text{-}1)$$

Illegal. On the right-hand side fact is not bound:
λ binds variables, not names. There is nothing there to refer back to.

Make the recursive call a parameter (abstract away):

$$G = \lambda f.\ \lambda n.\ n \leq 1\ ?\ 1 : n * f\ (n\text{-}1)$$

G contains no recursion at all. What we want back is an f with

$$f = G\ f \qquad \text{a } \textbf{fixed point} \text{ of } G$$

```
type IntFn = Int => Int

val G: IntFn => IntFn =
  f => n =>
    if n <= 1 then 1
    else n * f(n - 1)

val junk: IntFn = _ => 0

G(junk)(1)              // 1
G(G(junk))(2)           // 2
G(G(G(junk)))(3)        // 6
```

- Each application of G is correct one step further out, no matter what junk you start from.
- The fixed point is correct everywhere at once.
- Nothing so far is recursive — not the λ-term, not the code.
- Remaining question: how do you take a fixed point of something you cannot name?

Y combinator

A term can refer to itself if it is given a copy of itself. Write the copy as an argument:

$$M = \lambda x.\ G\ (x\ x)$$

$$M\ M \longrightarrow G\ (M\ M)$$

so M M is a fixed point of G. Abstract over G:

$$Y = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$$

$$Y\ G \longrightarrow G\ (Y\ G) \quad \textit{the fixed-point equation}$$

- x x applies a function to itself. The copy comes from substitution — nothing is duplicated in the source text.
- The two halves of Y are the same term. It is one idea, written twice.

…but under call-by-value:

to apply G you must first evaluate its argument, and the argument is M M:

$$\begin{aligned} Y\ G &\longrightarrow G\ (M\ M) \\ &\longrightarrow G\ (G\ (M\ M)) \\ &\longrightarrow G\ (G\ (G\ (M\ M))) \\ &\longrightarrow \ldots \end{aligned}$$

G is never actually applied.

Y is right for a lazy language

In a strict one — Scala, Java, ML, C — it is a loop. One small repair fixes it.

Z combinator: one η-expansion

The problem is $x\,x$ *in argument position.*
Eta-expand it:

$$x\,x \;\equiv\; \lambda v.\, x\,x\,v$$

$\lambda v.\, x\,x\,v$ is a lambda, hence a value. Evaluation stops there, and nothing unfolds until a real argument arrives.

$$Z = \lambda f.\, (\lambda x.\, f\,(\lambda v.\, x\,x\,v))\;\; (\lambda x.\, f\,(\lambda v.\, x\,x\,v))$$

$$Z\,G\,5 \;\longrightarrow\; 120$$

```
// x x needs a type T with T = T => A.
// No structural type has that shape; a
// named one does, and that is all it is for.
record Rec[A](unfold: Rec[A] => A)

val Z: (IntFn => IntFn) => IntFn = g => {
  val m = Rec[IntFn](x => v => g(x.unfold(x))(v))
  m.unfold(m)
}

val fact = Z(G)
fact(5)        // 120
```

μ — a fixed point for a record

An object refers to itself, so every object definition needs the same repair.

```
Empty = μ this. {
  isEmpty  = true,
  contains = λi. false,
  insert   = λi. Insert(this, i),
  union    = λs. s
}
```

and in a real language

```
// A record has many fields, so there is no
// single argument to eta-expand. Delay the
// whole self-reference instead:
//
//      x x   becomes   () => x x
//
// fix = λg. (λx. g (λ_. x x))
//           (λx. g (λ_. x x))

def fix[A](gen: (() => A) => A): A = {
  val m = Rec[A](x => gen(() => x.unfold(x)))
  m.unfold(m)
```

- Same Rec, same trick, one argument fewer.
- gen is the class: a function from the finished object to the finished object. fix is the one place the two ends are joined.

In a real object-oriented language you never write this — the runtime ties the knot at construction and calls the result this.

The objects: Empty and Insert

A class, here, is nothing but a function that returns a record.

```
val Empty: ISet = fix[ISet] { self =>
  ISet(
    isEmpty  = true,
    contains = _ => false,
    insert   = i => Insert(self(), i),
    union    = s => s            // empty ∪ s = s
  )
}

def Insert(s: ISet, n: Int): ISet =
  if (s.contains(n)) s           // already there
  else fix[ISet] { self =>
    ISet(
      isEmpty  = false,
      contains = i => i == n || s.contains(i),
      insert   = i => Insert(self(), i),
      union    = t => Union(self(), t)
    )
  }
```

- A class here is nothing but a function that returns a record. It is used to construct objects, never as a type.
- The “member variables” are the parameters s and n, captured by the closures
- `s.contains(i)` is a field selection followed by a call. We normally call that dynamic binding.
- Two optimisations are already visible:
  1. `empty ∪ s` returns s, and
  2. inserting a member returns the same object.
- No inheritance is used. Inheritance would remove the repetition in insert/union, *it is convenient, not essential*.

Union using objects

```
def Union(s1: ISet, s2: ISet): ISet = fix[ISet] { self =>
  ISet(
    isEmpty  = s1.isEmpty && s2.isEmpty,
    contains = i => s1.contains(i) || s2.contains(i),
    insert   = i => Insert(self(), i),
    union    = t => Union(self(), t)
  )
```

Everything it knows about s1 and s2, it learned by calling them.

The parameters are typed by the interface, so the representations of s1 and s2 are unreachable — even from inside the set library.

- Compare with C: there, union walked two linked lists and merged them in linear time.
- Here it cannot. No merge, no sorted order, no representation.
- Builds a small tree of pending questions: constructing the union is cheap, using costly.
- The C version optimises by looking inside. The object version stays flexible.
- That trade is the difference between the two kinds of abstraction.

The pure object contract

> *An object reaches other objects only through their public interfaces.*

- An object may know its own representation in every detail.
- Every other object is abstract to it.
- The converse is the useful half: any model that lets you inspect the representation of more than one abstraction at a time is not pure object-oriented.

Flexible abstraction

Nothing that already existed is edited, recompiled, or even read

```
val Even: ISet = fix[ISet] { self => ISet(
  isEmpty  = false,
  contains = i => i % 2 == 0,
  insert   = i => Insert(self(), i),
  union    = t => Union(self(), t)) }

val Full: ISet = fix[ISet] { self => ISet(
  isEmpty  = false,
  contains = _ => true,
  insert   = _ => self(),        // I am already everything
  union    = _ => self()) }

def Interval(n: Int, m: Int): ISet = fix[ISet] { self => ISet(
  isEmpty  = n > m,
  contains = i => n <= i && i <= m,
  insert   = i => Insert(self(), i),
  union    = t => Union(self(), t)) }
```

- Two of these sets are infinite. A sorted list cannot hold them
- Full answers every insert and every union with itself
- Interval stores two integers and computes membership.
- **This extensibility does not come from inheritance, it is a property of behavioural interfaces.**

Object interoperability

```
val a = Empty.insert(3).insert(1)
val b = a.union(Interval(10, 12)).union(Even)

b.contains(3)      // true   - from a
b.contains(11)     // true   - from the interval
b.contains(7)      // false  - nobody claims it
b.contains(1000)   // true   - from an infinite set
```

What we never used

Take away everything inessential, and see what is left.

`class as a type`

classes only construct

`inheritance`

reuse, not essence

`new`

a record literal suffices

`mutable state`

orthogonal to abstraction

`a dispatch table`

a record already is one

What is left is:

a record of closures, plus a fixed point for self.

Interface trade-offs

Try adding intersect : ISet → ISet to the interface.

```
def Intersect(s1: ISet, s2: ISet): ISet = fix[ISet] { self =>
  ISet(
    isEmpty   = ???,                  // no way to answer this
    contains  = i => s1.contains(i) && s2.contains(i),
    insert    = i => Insert(self(), i),
    union     = t => Union(self(), t),
    intersect = t => Intersect(self(), t)
  )
```

the standard fix costs flexibility

```
record ISet(
  ...
  iterator: () => Iterator[Int]   // Even cannot comply
)
```

- Looks easy, but isEmpty is not.
- Deciding whether an intersection is empty means enumerating one of the sets and testing each element, something the interface does not offer.
- The standard fix is to add an iterator. Finite sets now work but other (Even, Full and Primes) won't.

Good interfaces are hard to design.

Every method you add for efficiency's sake exposes something about the representation and narrows the space of legal implementations.

Optimisation

```
// 1. Empty.union is the identity function
val Empty: ISet = fix[ISet] { self =>
  ISet(isEmpty = true, contains = _ => false,
       insert  = i => Insert(self(), i),
       union   = s => s)                  // <- no work at all
}

// 2. Insert returns the set unchanged when n is a member
def Insert(s: ISet, n: Int): ISet =
  if s.contains(n) then s                 // <- no new object
  else fix[ISet] { self => /* ... */ }
```

With abstract data types you optimise across representations.
With objects you optimise locally.

- The sorted-list union from the C implementation has no object counterpart. To get it you would need a sorted-order iterator in the interface and then Even is no longer a set.
- But objects do get local optimisations, and the code we already wrote contains two.
- Empty.union is the identity function.
- Insert returns the existing set unchanged when the element is already present.
- That is why insert and union belong inside the interface rather than only as free-standing constructors: only the object knows its own cheap cases.

Verification

```
Bad  =  μ this . { isEmpty  = (random() > 0.5),
                   contains = λ i . (time() mod i = 1),
                   insert   = λ i . this,
                   union    = λ s . Insert(3, s) }
```

- Bad has exactly the right type. It is not a set.
- Type systems enforce structure, not specification.
- With an abstract data type, if the single implementation is correct then every client is correct and nobody can tamper with it.
- An object interface accepts any structurally compatible implementation, so no type system can keep Bad out.
- **You cannot tell what code will run** is the usual complaint about objects. It is the same complaint people make about higher-order functions, and it has the same cause: the code is a value, and values are chosen at run time.

"Object-oriented programming is designed to be as flexible as possible, almost as if it had been designed to be as difficult to verify as possible." -- William Cook

An object is a value whose entire public surface is a set of procedures.

Nothing else is required: no class, no inheritance, not even a type system.

- Objects hide with procedural abstraction, not type abstraction.
- Objects and their types are usually recursive.
- They can be read as closures, records of functions, first-class modules, or processes.
- They need no static type system at all — first-class functions are enough.

What makes a language object-oriented?

1

Multiple representations

the object, not the call site, chooses the code

2

Encapsulation

only the methods can touch the state

3

Subtyping

more methods may stand in for fewer

4

Inheritance

share method bodies between classes

5

Open recursion

self is bound when the object is built

Let's build all five from the material we already have: records of functions, and fix.

The object: a counter

The running example — the same shape as our sets, now with state.

```
Counter = {get: Unit→Nat, inc: Unit→Unit}

c = let x = ref 1 in
      { get = λ_:Unit. !x,
        inc = λ_:Unit. x := succ(!x) }
```

- A record of two functions
- The state is one mutable cell, captured by both functions.
- Objects are where state and behaviour get packaged together.

```
record Counter {
  get: () => Int,
  inc: () => Unit
}

c = {
  var x = 1              -- state: a cell
  Counter(
    get = () => x,
    inc = () => { x = x + 1 })
}

c.inc(); c.inc()
c.get()      -- 3
```

1 Multiple representations

The object, not the call site, decides which code runs.

```
counted = {
  var x = 1
  Counter(get = () => x, inc = () => { x = x + 1 })
}

tallied = {                          -- another representation
  var marks = "|"
  Counter(get = () => marks.length,
          inc = () => { marks += "|" })
}

tick = (c: Counter) => { c.inc(); c.get() }

tick(counted)     -- 2
tick(tallied)     -- 2
```

- One interface, two unrelated representations: an int and a string of tally marks.
- tick has no way to discover which it is holding, and no reason to want to.
- The call site does not pick the code. The object does.
- The same move as Even and Interval earlier, only now the objects have state.
- The methods are a group of behaviour, and different objects may answer the same message differently.

2 Encapsulation

The state is reachable only through the methods.

```
c = {
  var x = 1
  Counter(get = () => x, inc = () => { x = x +1 })
}

c.get()    -- 1
c.inc()
c.get()    -- 2

c.x        -- does not compile: there is no field x
```

Two different ways to hide the same integer

- An abstract data type hides its representation with a type, and the compiler enforces it.
- An object hides it with a scope, and there is nothing left to enforce — the name simply does not exist outside.
- A C struct counter { int x; } publishes x to everyone holding the pointer. That is why the C ADT needed the opaque-pointer trick.

- x is a local of a block that has already finished running. The only two references to it left anywhere are the two functions in the record.
- Nothing else can name it, so nothing else can break it: the invariant is whatever those two bodies maintain.
- In λ-calculus notation the state is bound by let x = ref 1 in … the scope of the binding is the encapsulation.
- When the hidden state is itself made of objects, references to it leak out through the methods that return them. Ownership types can help with this.

Object generators, and what a class is

Two refactorings that the next two features cannot do without.

```
-- 1. an object generator: a fresh counter on every call
newCounter = () => {
  var x = 1
  Counter(get = () => x, inc = () => { x = x + 1 })
}

-- 2. group the instance variables into one record
record CounterRep { var x: Int }

-- 3. a class = a function from representation to methods
counterClass = (r: CounterRep) =>
  Counter(
    get = () => r.x,
    inc = () => { r.x = r.x + 1 })

-- "new" = allocate a representation, then apply the class
newCounter2 = () => counterClass(CounterRep(x = 1))
```

- A class is not a type and not a language primitive. It is a function from a representation to a record of methods.
- new is now visible as two steps:
  1. allocate the representation
  2. apply the class to it.
- Why? We will need it for method reuse and one cannot reuse a body that is welded to one particular closure.

3 Subtyping

A record with more fields may stand in for one with fewer.

```
record ResetCounter {
  get:   () => Int,
  inc:   () => Unit,
  reset: () => Unit
}

rc = resetCounterClass(CounterRep(x = 1))

tick(rc)          -- a ResetCounter where a Counter is
                  -- wanted. No coercion. No copy. No cost.
```

Record width subtyping:

{get: Unit→Nat, inc: Unit→Unit, reset: Unit→Unit}
<:
{get: Unit→Nat, inc: Unit→Unit}

S <: T means an S may stand in wherever a T is expected.

- Without the rule you would have to forget the extra field by hand:

```
asCounter = (rc: ResetCounter) =>
  Counter(rc.get, rc.inc)

tick(asCounter(rc))
```

- Subtyping is exactly that coercion, supplied by the type system and free at run time, because the wider record already holds the fields.
- It is also what lets representations with different interfaces be used together.

4 Inheritance

Sharing method bodies, with no keyword in sight.

```
resetCounterClass = (r: CounterRep) => {
  sup = counterClass(r)          -- "super" is a value
  ResetCounter(
    get   = sup.get,             -- inherited
    inc   = sup.inc,             -- inherited
    reset = () => { r.x = 1 })   -- new
}

newResetCounter = () =>
  resetCounterClass(CounterRep(x = 1))
```

- Inheritance is a function call and a record copy. No keyword, no magic.
- The subclass shares the representation, the same CounterRep, and the method bodies.
- super is an ordinary value here. Elsewhere it is a name the compiler resolves
- Overriding means putting a different function in a field instead of copying super's.

but it is still broken

If a subclass overrides inc, the copies taken from sup still run the old code. sup's functions captured r, the representation, not the finished object. Fixing that is the next feature.

5 Open recursion — self becomes a parameter

A method body has to be able to call the finished object's methods.

```
record SetCounter {
  get: () => Int, set: Int => Unit, inc: () => Unit
}

-- inc should be written in terms of set and get. Whose set?
--   ours          -> a subclass that overrides set is ignored
--   the object's -> it does not exist yet
-- Answer: take self as a parameter, close the loop later.

setCounterClass = (r: CounterRep) => (self: () => SetCounter) =>
  SetCounter(
    get = () => r.x,
    set = i  => { r.x = i },
    inc = () => self().set(self().get() + 1))    -- late bound

newSetCounter = () => {
  r = CounterRep(x = 1)          -- the cell is made ONCE,
  fix(self => setCounterClass(r)(self))    -- outside the knot
}
```

- The class leaves the recursion open: it is a function of self.
- fix closes it when the object is created — the very same fix we used for sets, the same μ.
- “Open” is the operative word. Between writing the class and building the object, someone else may change what self means.
- A real object-oriented language runs this fix for you at construction time and calls the result this.
- The sets in Part I were already doing it: insert = i => Insert(self(), i).
- **A class is an open recursion; an object is its fixed point.**

Open recursion in action

An inherited method changes behaviour although nobody touched it.

```
record InstrRep     { var x: Int, var a: Int }    -- <: CounterRep

record InstrCounter {
  get: () => Int,   set:      Int => Unit,
  inc: () => Unit,  accesses: () => Int            -- <: SetCounter
}

instrCounterClass = (r: InstrRep) => (self: () => InstrCounter) => {
  sup = setCounterClass(r)(self)     -- both widenings are free
  InstrCounter(
    get      = sup.get,                            -- inherited
    set      = i => { r.a = r.a + 1; sup.set(i) },  -- overridden
    inc      = sup.inc,                            -- inherited!
    accesses = () => r.a)
}

newInstrCounter = () => {
  r = InstrRep(x = 1, a = 0)
  fix(self => instrCounterClass(r)(self))
}

ic = newInstrCounter()
ic.inc()
ic.get()        -- 2
ic.accesses()   -- 1   <- the inherited inc ran the OVERRIDDEN set
```

- inc was inherited without a single change, yet it now counts accesses.
- Because sup was built with our self, every call it makes to set lands on the overriding version.
- That is open recursion. It is also the whole of late binding and dynamic dispatch.
- And it is why inheritance is fragile: change which methods a superclass calls on self, and every subclass silently changes behaviour.

Add an operation: intersect

abstract data type — two additions

```c
/* set.h */
set set_intersect(set s1, set s2);      /* + declaration */

/* set.c */
set set_intersect(set s1, set s2) {     /* + definition  */
  if (s1 == NULL || s2 == NULL) return NULL;
  if (s1->head < s2->head)
    return set_intersect(s1->tail, s2);
  if (s1->head > s2->head)
    return set_intersect(s1, s2->tail);
  return cons(s1->head, set_intersect(s1->tail, s2->tail));
}

/* Nothing else in the program changes, and every set
   that already exists gains the operation for free. */
```

objects — every implementation

```
final case class ISet(
  isEmpty:   Boolean,
  contains:  Int => Boolean,
  insert:    Int => ISet,
  union:     ISet => ISet,
  intersect: ISet => ISet     // + one field
)

// ...and now every implementation
// stops compiling: Empty, Insert,
// Union, Even, Full, Interval,
// plus every set anybody else ever
// wrote against the old interface.
// Plus some things are no longer
// representable such as Even, Full, …
```

An abstract data type groups the code by operation, so a new operation is one new self-contained block.

An object groups the code by representation, so a new operation is a cut straight across every one of them.

Add a representation: the even numbers

abstract data type — the representation itself changes

```c
/* set.c */
struct set_rep {
  enum { LIST, EVENS } tag;          /* + a tag         */
  int head; struct set_rep *tail;    /* only when LIST */
};

bool contains(set s, int i) {
  switch (s->tag) {                  /* + a branch      */
    case EVENS: return i % 2 == 0;
    case LIST:  /* ...as before... */ ;
  }
}

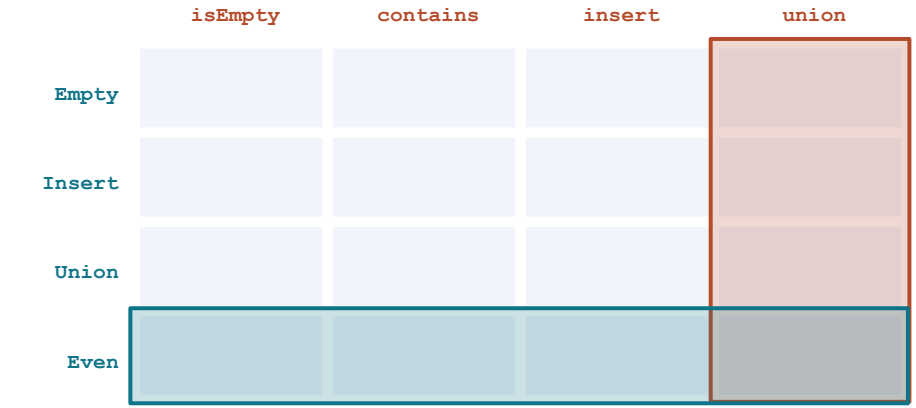
/* the same again in is_empty, insert, set_union and
   set_intersect — and set_union needs one branch per
   PAIR of variants. Everything that links against
   set.c is recompiled, including code you do not own. */
```

objects — one new value

```
val Even: ISet = fix[ISet] { self =>
  ISet(
    isEmpty  = false,
    contains = i => i % 2 == 0,
    insert   = i => Insert(self(), i),
    union    = t => Union(self(), t))
}

// Done.
//
// Nothing else is edited.
// Nothing else is recompiled.
// Nobody else needs to be told.
```

The extensibility problem



an object groups by row:
one representation, all operations

an ADT groups by column:
one operation, all representations

- Organise the behaviour as a matrix: representations down one axis, operations across the other.
- Extending means adding a row or a column. Each style makes one of them easy and the other painful.
- Reynolds in 1975: user-defined types favour new operations. Procedural data structures favour new representations. He was describing objects without using the word.
- The duality has been rediscovered at least three times since. Multi-methods, visitors, type classes and tagless-final styles are all attempts on it.
- Nobody has a fully satisfying answer for independently developed extensions.

What objects cost

Operations you cannot write

intersect needs to enumerate a set. Add an iterator and infinite sets break.

The interface you choose decides which operations are fast, slow, or impossible.

Optimisation pressure

To get the fast union back, publish “iterate in sorted order”.

Every such method buys speed by spending flexibility the representation leaks into the interface.

Nothing comes for free

No equality, no printing, no serialisation, unless the interface provides them.

Autognosis takes away exactly the tools that would have supplied them.

this type-checks

```
val Bad: ISet = fix[ISet] { self => ISet(
  isEmpty  = math.random() > 0.5,
  contains = i => i != 0 && System.nanoTime() % i == 1,
  insert   = _ => self(),
  union    = s => Insert(s, 3)) }
```

- A structural interface cannot rule out nonsense. Only a behavioural specification can.
- Verification is hard for the same reason first-class functions are hard plus re-entrancy: a method can be called back while its own invariant is broken.
- When an object's state is made of other objects, it tends to leak out. Hence ownership types.

Real languages blend the two

objects in C — no keyword required

```c
typedef struct ISet {
  void *rep;                          /* mine alone */
  bool (*contains)(struct ISet*, int);
  struct ISet *(*insert)(struct ISet*, int);
  struct ISet *(*join)(struct ISet*, struct ISet*);
} ISet;

s->contains(s, 42);   /* dispatch: pick and call */z
```

- struct file_operations in the Linux kernel, GObject, COM vtables, FILE* all objects, written in C.
- The pattern is a record of function pointers plus a self argument. That is the whole mechanism.

ADTs in Java — no keyword required

```java
class ASet {
  // representation fields, no public ctor
  static ASet empty();
  static ASet insert(ASet s, int n);
  static boolean contains(ASet s, int n);
  static ASet union(ASet a, ASet b);
}
// a class name used as a type = type abstraction
```

- The moment a class name is used as a type, you have chosen an abstract data type.
- Three rules that force a pure object-oriented style in Java: class names only after new; only interfaces as types; no ==, no casts, no instanceof.

Intermezzo: Smalltalk

There is no built-in control flow. There is only message passing.

Smalltalk

```
class True
    ifTrue: a ifFalse: b
        ^ a value

class False
    ifTrue: a ifFalse: b
        ^ b value


(x > y) ifTrue: [ x print ] ifFalse: [ y print ]
```

- There are two Boolean classes, True and False, and both implement a single method, ifTrue:ifFalse:
- True returns the first argument, False returns the second. That is the entire conditional.
- [...] is a block (a thunk). Sending value message forces it.
- So you can write a RandomBoolean, or a LoggingBoolean that counts how often it was asked and pass either one anywhere a boolean goes, including into system code.

Control flow is not part of the language. It is a method, and the receiver decides which branch runs.

That is Church encoding

The same construction, thirty years earlier.

λ-calculus (Church, 1941):

```
true   =  λ t . λ f . t
false  =  λ t . λ f . f

if b then x else y   =   b x y
```

The value is its own conditional.

The untyped λ-calculus was the first object-oriented language.**

References

The framing of this lecture — that objects and abstract data types are two different kinds of data abstraction, not two spellings of one It build on the work of William Cook, On Understanding Data Abstraction, Revisited (OOPSLA 2009) and Pierce's, Types and Programming Languages, chapter 18.

J. C. Reynolds, 1975
User-defined types and procedural data structures as complementary approaches to data abstraction — the original insight.

L. Cardelli & P. Wegner, 1985
On understanding types, data abstraction, and polymorphism — the survey the 2009 essay answers.

J. Mitchell & G. Plotkin, 1985

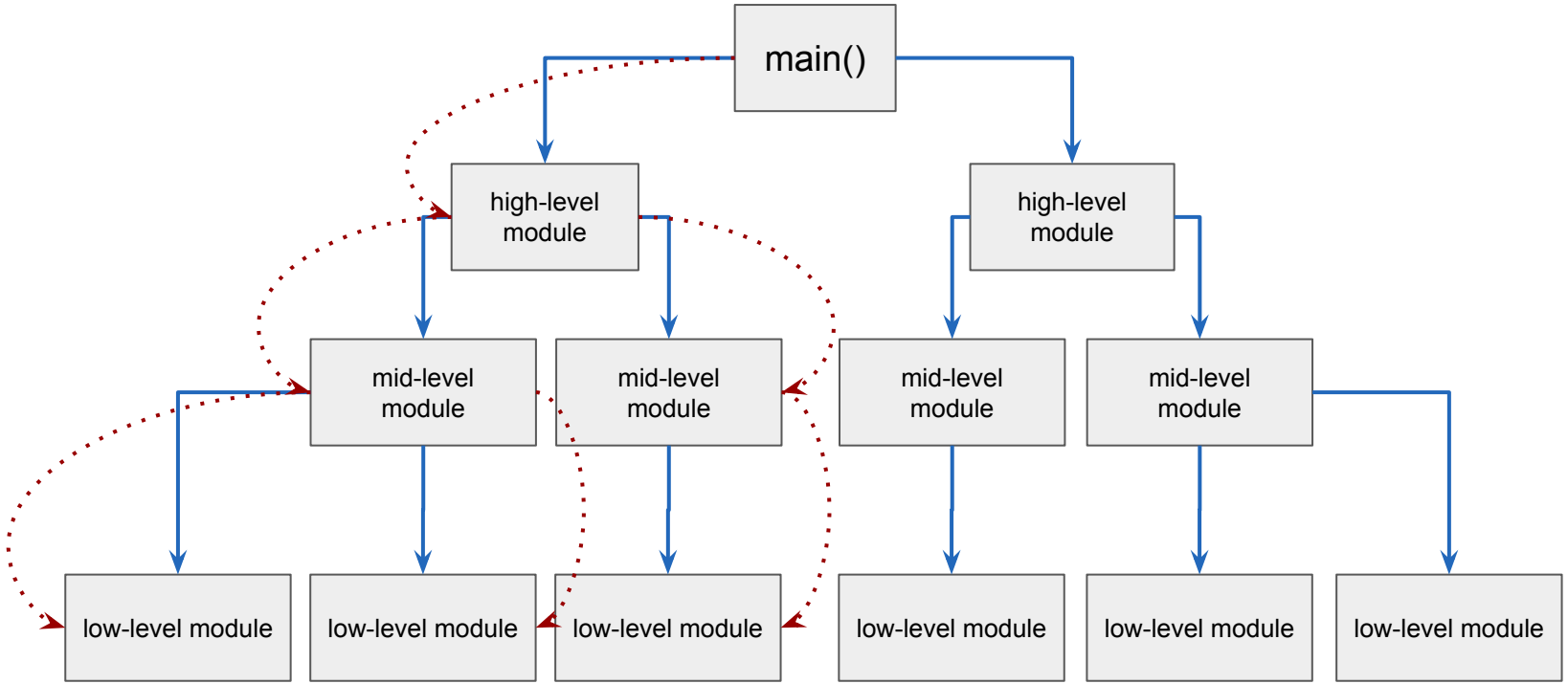
Abstract types have existential type.

W. Cook, 1989 / 1990
A Denotational Semantics of Inheritance (thesis); Object-oriented programming versus abstract data types.

B. Pierce, 2002

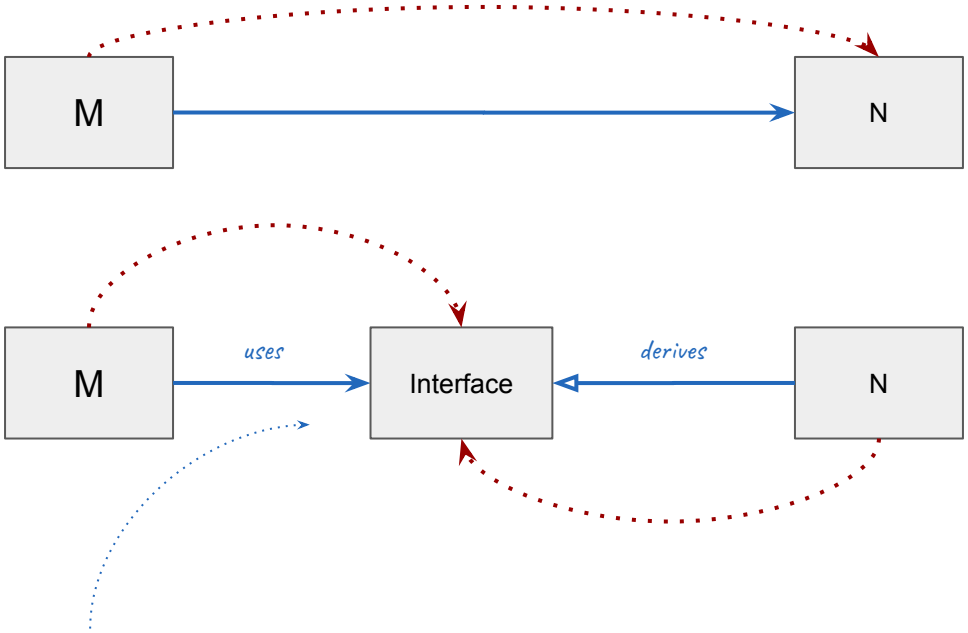
Types and Programming Languages — objects in ch. 18, existential types in ch. 24.

Another look at the dependencies



The dependencies follow the control

Another look at the dependencies

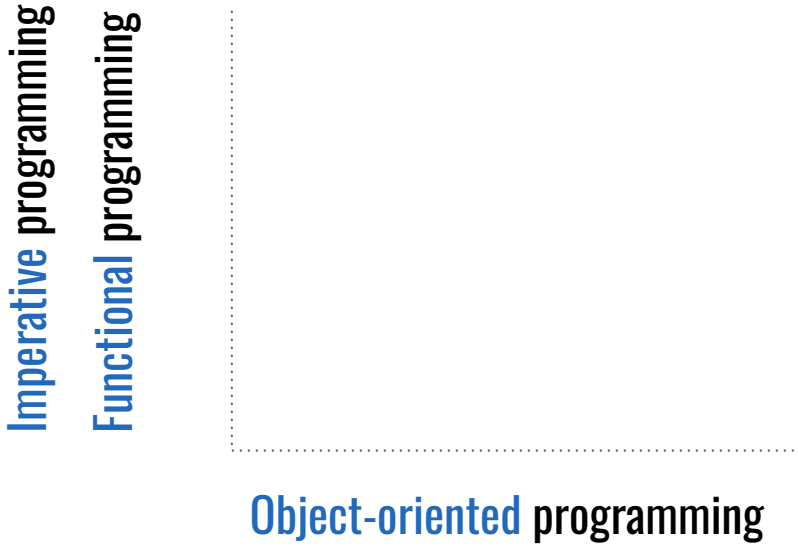


Polymorphism which allows us to reverse the dependencies!
Decouple M from N

Summary

- Programming takes discipline
- OOP helps us with it by giving us ways to
  - organize code so we can write cohesive code
  - reuse code via inheritance so we can write less code
  - manage dependencies so we can decouple code
- It is a double-edged sword
  - With every new layer of abstraction we introduce a maintenance and performance cost

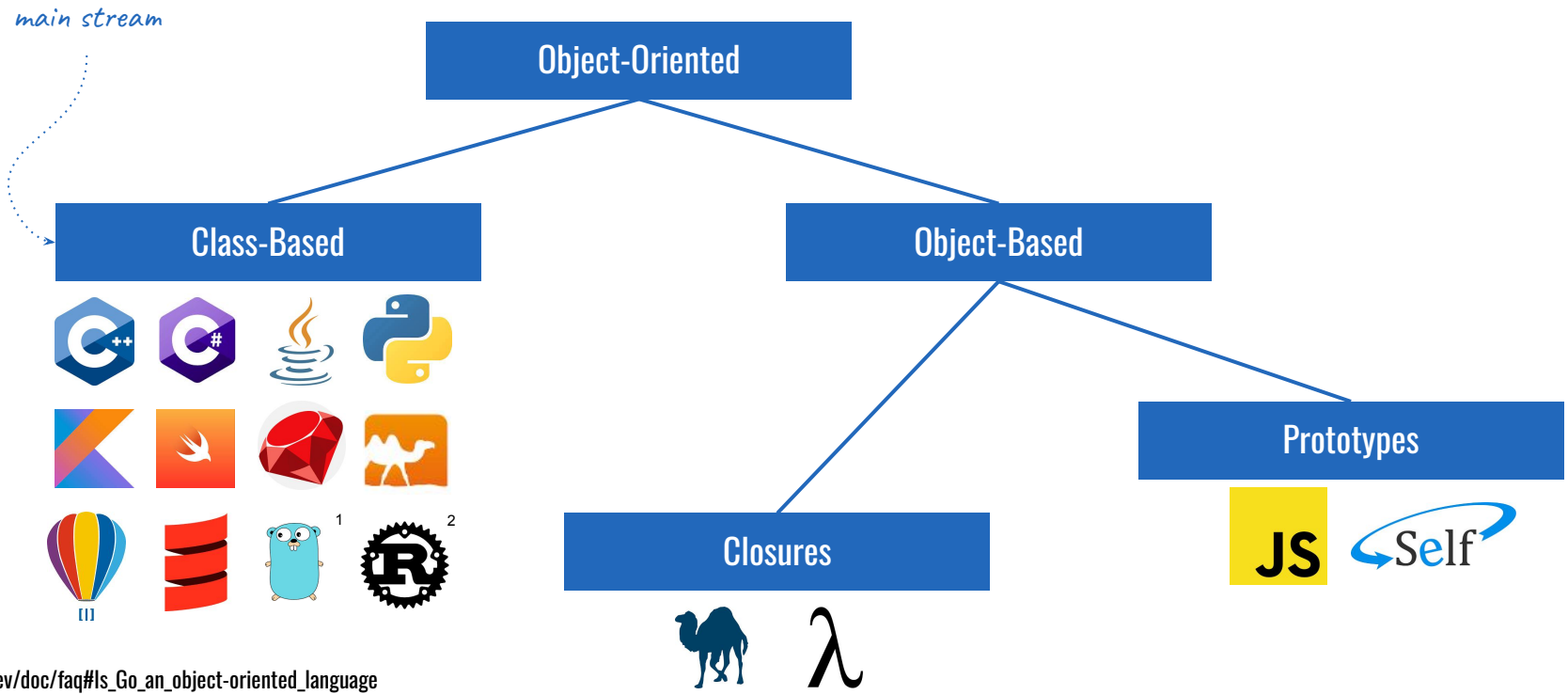
Programming paradigms



"Object-orientation is a dimension orthogonal to the imperative/functional dimension."

-- Martin Odersky

Hierarchy of OOP

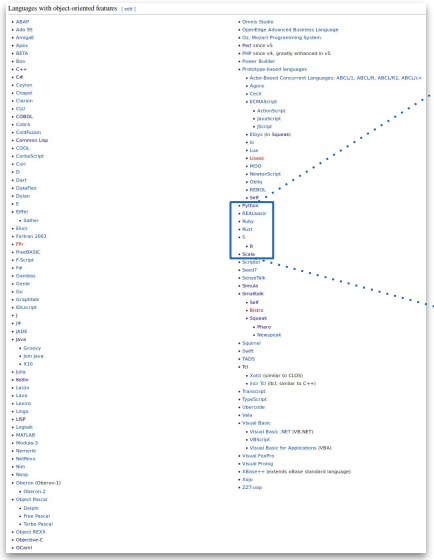


[1]https://go.dev/doc/faq#Is_Go_an_object-oriented_language

[2]https://doc.rust-lang.org/book/ch17-00-oop.html

Object-oriented programming languages

- Practical OOP



- Python
- REALbasic
- Ruby
- Rust
- S
  - R
- Scala

Scala

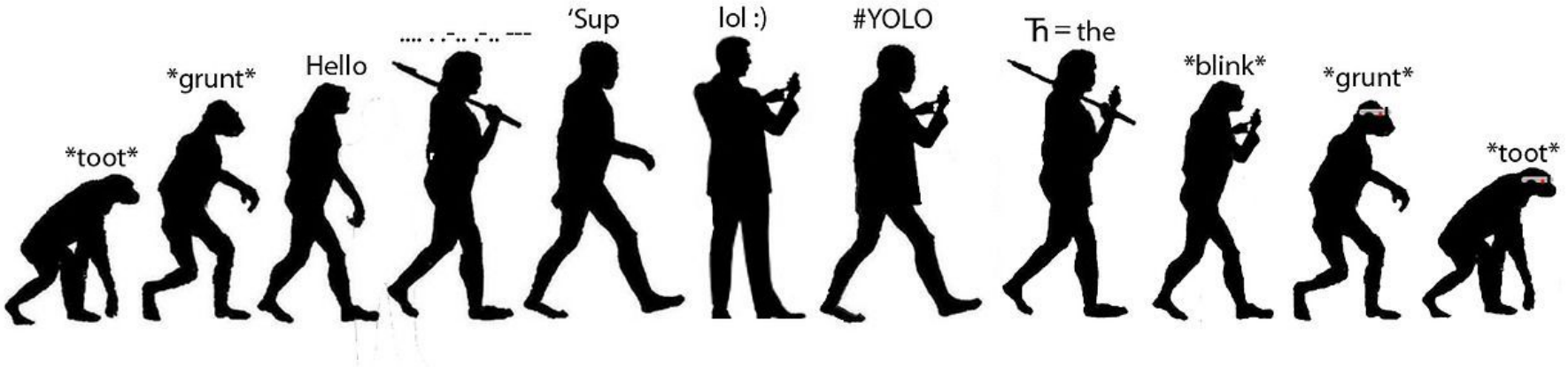
- Modern strongly typed general-purpose programming language
- Practical OOP and FP support
- Runs on JVM (JS, native)

- PL for the labs/project

https://en.wikipedia.org/wiki/List_of_object-oriented_programming_languages

Why Scala?

Language evolution



https://www.pinterest.com/pin/the-evolution-of-language--199143614746195509/

```
List filtered = new ArrayList();

for (int i = 0; i < students.size(); i++) {
    Object o = students.get(i);
    if (o instanceof Student) {
        Student s = (Student) o;
        if (s.getGpa() >= HIGH_GPA) {
            filtered.add(s.getName());
        }
    }
}
```

Java 1.4

Language evolution

```
List filtered = new ArrayList();

for (int i = 0; i < students.size(); i++) {
    Object o = students.get(i);
    if (o instanceof Student) {
        Student s = (Student) o;
        if (s.getGpa() >= HIGH_GPA) {
            filtered.add(s.getName());
        }
    }
}
```

Java 1.4

```
List<String> filtered = new ArrayList<String>();

for (Student s : students) {
   if (s.getGpa() >= HIGH_GPA) {
       filtered.add(s.getName());
   }
}
```

generics

enhanced for loop

Java 5

Language evolution

```
List filtered = new ArrayList();

for (int i = 0; i < students.size(); i++) {
    Object o = students.get(i);
    if (o instanceof Student) {
        Student s = (Student) o;
        if (s.getGpa() >= HIGH_GPA) {
            filtered.add(s.getName());
        }
    }
}
```

Java 1.4

generics

```
List<String> filtered = new ArrayList<String>();

for (Student s : students) {
   if (s.getGpa() >= HIGH_GPA) {
       filtered.add(s.getName());
   }
}
```

enhanced for loop

Java 5

diamond operator

```
List<String> filtered = new ArrayList<>();

for (Student s : students) {
   if (s.getGpa() >= HIGH_GPA) {
       filtered.add(s.getName());
   }
}
```

Java 7

Language evolution

```
List filtered = new ArrayList();

for (int i = 0; i < students.size(); i++) {
    Object o = students.get(i);
    if (o instanceof Student) {
        Student s = (Student) o;
        if (s.getGpa() >= HIGH_GPA) {
            filtered.add(s.getName());
        }
    }
}
```

Java 1.4

generics

```
List<String> filtered = new ArrayList<String>();

for (Student s : students) {
   if (s.getGpa() >= HIGH_GPA) {
       filtered.add(s.getName());
   }
}
```

enhanced for loop

Java 5

diamond operator

```
List<String> filtered = new ArrayList<>();

for (Student s : students) {
   if (s.getGpa() >= HIGH_GPA) {
       filtered.add(s.getName());
   }
}
```

Java 7

local variable type inference

Stream API

```
var filtered =
    students
      .stream()
      .filter(s -> s.getGpa() >= HIGH_GPA)
      .map(s -> s.getName())
      .collect(toList());
```

lambda

Java 10



```java
var filtered =
    students
      .stream()
      .filter(s -> s.getGpa() >= HIGH_GPA)
      .map(Student::getName)
      .collect(toList());
```



```cpp
std::vector<std::string> filtered;

for (size_t i = 0; i < students.size(); ++i) {
  const Student& s = students[i];
  if (s.getGpa() >= HIGH_GPA) {
    filtered.push_back(s.getName());
  }
}
```

```cpp
std::vector<std::string> filtered;

std::for_each(students.begin(), students.end(),
  [&](const Student& s) {
    if (s.getGpa() >= HIGH_GPA) {
      filtered.push_back(s.getName());
    }
  }
);
```



```
var filtered =
    students
      .stream()
      .filter(s -> s.getGpa() >= HIGH_GPA)
      .map(Student::getName)
      .collect(toList());
```

2018



```
val filtered =
    students.filter(_.gpa >= HIGH_GPA)
            .map(_.name)
```

2006

Why Scala?

- Ergonomic and concise syntax (less noise, less typing, better fits on slides)
- Once you understand these concepts once, you will be able to grasp them in other languages
- The more languages you experiment with, the better
- Interesting language worth studying
  - Interesting blend between FP and OOP
  - Algebraic data types and pattern matching
  - Higher-order functions
  - Expressive collection operations
  - Advanced type system
    - Traits, union typed, intersection types, higher-kinder types, ...
  - Type classes and extensions

LAMBDA WORLD
CÁDIZ by 47 Degrees

Scala Implicits are Everywhere

A large-scale study of the use of Implicits in the wild

(OOPSLA 2019)

FILIP KŘIKAVA, Czech Technical University in Prague, CZ
HEATHER MILLER, Carnegie Mellon University, USA
JAN VITEK, Czech Technical University in Prague and Northeastern University, USA



At a glance:

- 7,280 Scala projects
- 18M lines of code
- 8M implicit call sites
- 370K implicit declarations

98% projects use implicits

78% of projects define implicits

27% call sites use implicits

The top of the graph shows the ratio of call sites, in each project, that involves implicit resolution. The bottom shows the number of implicit definitions in each project (log scale).

Packlink' Simply Shipping · THE WORKSHOP™ · ThoughtWorks · SIGNIFY TECHNOLOGY · LUNATECH · RavenPack · letgo · Tecalis · ZONAFRANCA CÁDIZ

Thank you